

Algorithmic Trading System API Documentation

Introduction

The Algorithmic Trading System API provides a robust interface for interacting with a comprehensive trading platform that supports a wide range of asset classes, including stocks, ETFs, futures, options, forex, commodities, and cryptocurrencies. This API enables users to retrieve market data, place and manage trades, run backtests, and manage trading strategies programmatically. Designed for both paper and live trading, the API abstracts data source complexities, ensuring high availability and seamless integration with Interactive Brokers for live trading and multiple free sources for paper trading.

This documentation outlines the API's endpoints, authentication methods, request and response formats, error handling, and usage guidelines, along with examples to help you get started.

Authentication

The API uses JSON Web Tokens (JWT) for authentication. To access the API, you must include a valid token in the `Authorization` header of each request.

Obtaining a Token

1. Log in to the trading system via the web interface or authentication endpoint.
2. Generate a token from your account settings or use the `/auth/token` endpoint (not documented here as it's part of the user management system).
3. Store the token securely for use in API requests.

Using the Token

Include the token in the `Authorization` header as a Bearer token:

Authorization: Bearer <your_token_here>

Permissions

Certain endpoints require specific user roles (e.g., Admin, Trader, Quant, Risk Manager). Attempting to access an endpoint without the required permissions will result in a `403 Forbidden` error.

Endpoints

All endpoints are prefixed with `/v1/` to indicate the API version. Below are the main categories of endpoints:

Data Endpoints

Get Market Data

- **URL:** `/v1/data`
- **Method:** `GET`
- **Description:** Retrieves market data for a specified asset class and symbol. Supports both historical and real-time data.
- **Parameters:**
 - `asset_class` (required, string): The asset class (e.g., `stock`, `option`, `future`, `forex`, `commodity`, `crypto`).
 - `symbol` (required, string): The asset symbol (e.g., `AAPL`, `ESZ23`, `BTCUSD`).
 - `type` (optional, string): Data type to retrieve (e.g., `price`, `volume`, `open_interest`). Defaults to `price`.
 - `start` (optional, string): Start date for historical data (ISO 8601, e.g., `2023-01-01`).
 - `end` (optional, string): End date for historical data (ISO 8601, e.g., `2023-12-31`).

- `real_time` (optional, boolean): If `true`, returns the latest data point; ignores `start` and `end`. Defaults to `false`.
- Asset-specific parameters:
 - For option: `expiration` (string, e.g., `2024-01-19`), `strike` (number, e.g., `150.0`), `option_type` (string, `call` or `put`).
 - For future: `contract_month` (string, e.g., `Z23` for December 2023).

- **Response:**

- Historical: JSON array of data points with `timestamp` and requested `type`.
- Real-time: JSON object with the latest data point.

- **Example Request (Historical Stock Data):**

```
GET /v1/data?asset_class=stock&symbol=AAPL&type=price&start=2023-01-01
Authorization: Bearer <token>
```



- **Example Response:**

```
[
  { "timestamp": "2023-01-01T00:00:00Z", "price": 150.00 },
  { "timestamp": "2023-01-02T00:00:00Z", "price": 152.50 }
]
```

- **Example Request (Real-Time Option Data):**

```
GET /v1/data?asset_class=option&symbol=AAPL&expiration=2024-01-19&stri
Authorization: Bearer <token>
```



- **Example Response:**

```
{
  "timestamp": "2023-10-15T14:30:00Z",
  "price": 10.50,
  "bid": 10.40,
```

```
    "ask": 10.60,  
    "iv": 0.25,  
    "delta": 0.6  
  }  
}
```

Get Options Chain

- **URL:** /v1/data/options_chain
- **Method:** GET
- **Description:** Retrieves the options chain for a specified symbol and date, including prices, greeks, and implied volatility.
- **Parameters:**
 - **symbol** (required, string): The underlying symbol (e.g., AAPL).
 - **date** (required, string): The date for the options chain (ISO 8601, e.g., 2023-12-31).
- **Response:** JSON object with `calls` and `puts` arrays, each containing option contract details.
- **Example Request:**

```
GET /v1/data/options_chain?symbol=AAPL&date=2023-12-31  
Authorization: Bearer <token>
```

- **Example Response:**

```
{  
  "calls": [  
    {  
      "strike": 150.0,  
      "expiration": "2023-12-31",  
      "last_price": 10.50,  
      "bid": 10.40,  
      "ask": 10.60,  
      "volume": 1000,  
      "open_interest": 5000,  
      "iv": 0.25,  
      "delta": 0.6,  
      "gamma": 0.03,  
    }  
  ]  
}
```

```

        "theta": -0.02,
        "vega": 0.15
    }
],
"puts": [
    {
        "strike": 150.0,
        "expiration": "2023-12-31",
        "last_price": 5.20,
        "bid": 5.10,
        "ask": 5.30,
        "volume": 800,
        "open_interest": 3000,
        "iv": 0.30,
        "delta": -0.4,
        "gamma": 0.02,
        "theta": -0.01,
        "vega": 0.12
    }
]
}

```

Trading Endpoints

Place Order

- **URL:** /v1/orders
- **Method:** POST
- **Description:** Places a new trading order.
- **Request Body:**

```

{
    "asset_class": "stock",
    "symbol": "AAPL",
    "type": "market",
    "side": "buy",
    "quantity": 10,
    "price": null
}

```

- **asset_class** (required, string): Asset class of the order.

- `symbol` (required, string): Asset symbol.
 - `type` (required, string): Order type (`market` , `limit`).
 - `side` (required, string): `buy` or `sell` .
 - `quantity` (required, number): Number of units to trade.
 - `price` (optional, number): Price for limit orders; `null` for market orders.
- **Response:** JSON object with order details.
 - **Example Response:**

```
{
  "order_id": "12345",
  "status": "submitted"
}
```

Cancel Order

- **URL:** `/v1/orders/cancel`
- **Method:** `POST`
- **Description:** Cancels an existing order.
- **Request Body:**

```
{
  "order_id": "12345"
}
```

- **Response:** JSON object confirming cancellation.
- **Example Response:**

```
{
  "order_id": "12345",
  "status": "cancelled"
}
```

Get Order Status

- **URL:** /v1/orders/{order_id}
- **Method:** GET
- **Description:** Retrieves the status of a specific order.
- **Path Parameters:**
 - order_id (required, string): The ID of the order.
- **Response:** JSON object with order details.
- **Example Request:**

```
GET /v1/orders/12345
Authorization: Bearer <token>
```

- **Example Response:**

```
{
  "order_id": "12345",
  "status": "filled",
  "symbol": "AAPL",
  "quantity": 10,
  "filled_quantity": 10,
  "avg_fill_price": 152.50
}
```

Get Portfolio

- **URL:** /v1/portfolio
- **Method:** GET
- **Description:** Retrieves the current portfolio holdings and values.
- **Response:** JSON object with cash balance and positions.
- **Example Request:**

```
GET /v1/portfolio
Authorization: Bearer <token>
```

- **Example Response:**

```
{
  "cash": 10000.00,
  "positions": [
    {
      "symbol": "AAPL",
      "quantity": 10,
      "avg_price": 150.00,
      "market_value": 1520.00
    }
  ],
  "total_value": 11520.00
}
```

Strategy Endpoints

Run Backtest

- **URL:** /v1/backtest
- **Method:** POST
- **Description:** Runs a backtest for a specified strategy.
- **Request Body:**

```
{
  "strategy_id": "my_strategy",
  "start_date": "2023-01-01",
  "end_date": "2023-12-31",
  "initial_capital": 100000.00
}
```

- **Response:** JSON object with backtest performance metrics.
- **Example Response:**

```
{
  "sharpe_ratio": 1.5,
  "max_drawdown": 0.10,
```

```
    "total_return": 0.20
}
```

Error Handling

The API uses standard HTTP status codes and includes detailed error messages in the response body.

Common Error Codes

- 400 Bad Request : Invalid parameters or malformed request.
- 401 Unauthorized : Missing or invalid authentication token.
- 403 Forbidden : Insufficient permissions for the requested action.
- 404 Not Found : Resource (e.g., order, symbol) not found.
- 429 Too Many Requests : Rate limit exceeded.
- 500 Internal Server Error : Unexpected server issue.

Example Error Response

```
{
  "error": {
    "code": 401,
    "message": "Invalid or expired token"
  }
}
```

Data Reliability and Fallback Mechanism

The system ensures high data availability through a sophisticated fallback mechanism for paper trading and direct integration with Interactive Brokers for live trading.

Paper Trading Data Sources

For paper trading, the system uses free data sources with a fallback mechanism. If a primary source fails, it switches to the next source in the list:

- **Stocks & ETFs:** Interactive Brokers → Yahoo Finance → Alpha Vantage → Finnhub → Twelve Data → Polygon.io
- **Stock Futures & Index Futures:** Interactive Brokers → Yahoo Finance → Investing.com → CME Group → Barchart
- **Stock Options & Index Options:** Interactive Brokers → Yahoo Finance → Cboe Daily Market Statistics → SpiderRock
- **Forex, Forex Futures, & Forex Options:** Interactive Brokers → Yahoo Finance → Oanda → CME Group FX Data → dxFeed
- **Commodities, Commodity Futures, & Commodity Options:** Interactive Brokers → Yahoo Finance → TradingCharts → CME Group
- **Cryptocurrencies:** Interactive Brokers → Alpha Vantage → Coinbase → Binance India

Live Trading Data Sources

For live trading, the system subscribes to Interactive Brokers' real-time market data, providing low-latency and accurate data.

Options Data Features

- **Historical Options Chain Data:** Minimum depth of 5 years.
- **Real-Time Implied Volatility Surfaces:** Available via the `/v1/data` endpoint with appropriate parameters.
- **Dividend Forecast Integration:** Incorporated into options pricing and analytics.

The API abstracts these details, providing a consistent interface regardless of the underlying source or trading mode (paper or live).

Rate Limiting

To ensure fair usage and compliance with external data provider limits, the API enforces the following rate limits:

- 100 requests per minute per user

- 10,000 requests per day per user

Monitoring Usage

Response headers include:

- `X-RateLimit-Limit` : Maximum requests allowed in the current window.
- `X-RateLimit-Remaining` : Remaining requests in the current window.
- `X-RateLimit-Reset` : Timestamp when the window resets (Unix epoch).

Exceeding the limit returns a `429 Too Many Requests` error. Contact support for higher limits.

Versioning

The API uses a version prefix (e.g., `/v1/`) in the URL. Future updates will increment the version (e.g., `/v2/`) to avoid breaking existing integrations.

Changelog

Version 1.0 (Current)

- Initial release with trading, data, and strategy endpoints.
- Support for all asset classes and paper/live trading modes.

Getting Started

Making Your First API Call

1. **Obtain a Token:** Log in and generate a token from your account settings.
2. **Set Up Headers:** Include the token in the `Authorization` header.
3. **Make a Request:** Use a simple GET request to retrieve data.

Example (Python)

```
import requests

token = "your_token_here"
headers = {"Authorization": f"Bearer {token}"}
params = {"asset_class": "stock", "symbol": "AAPL", "real_time": "true"}
response = requests.get("https://api.trading-system.com/v1/data", headers=headers, params=params)
print(response.json())
```

Expected Output:

```
{
  "timestamp": "2023-10-15T14:30:00Z",
  "price": 152.50
}
```

Examples

Retrieve Historical Options Data (JavaScript)

```
const fetch = require('node-fetch');

const token = 'your_token_here';
const headers = { 'Authorization': `Bearer ${token}` };
const url = 'https://api.trading-system.com/v1/data?asset_class=option&symbol=AAPL';

fetch(url, { headers })
  .then(response => response.json())
  .then(data => console.log(data))
  .catch(error => console.error(error));
```

Place a Market Order (Python)

```
import requests

token = "your_token_here"
headers = {"Authorization": f"Bearer {token}"}
```

```
data = {
    "asset_class": "stock",
    "symbol": "AAPL",
    "type": "market",
    "side": "buy",
    "quantity": 10,
    "price": None
}
response = requests.post("https://api.trading-system.com/v1/orders", headers=headers, data=json.dumps(data))
print(response.json())
```

Supported Asset Classes

- **Stocks & ETFs:** `asset_class=stock` , e.g., `AAPL` .
- **Futures:** `asset_class=future` , e.g., `ESZ23` (E-mini S&P 500 Dec 2023).
- **Options:** `asset_class=option` , requires `expiration` , `strike` , `option_type` .
- **Forex:** `asset_class=forex` , e.g., `EURUSD` .
- **Commodities:** `asset_class=commodity` , e.g., `CL` (crude oil).
- **Cryptocurrencies:** `asset_class=crypto` , e.g., `BTCUSD` .

Data Formats and Conventions

- **Dates:** ISO 8601 format (e.g., `2023-12-31T00:00:00Z`).
- **Numbers:** Prices and quantities use up to 2 decimal places unless specified otherwise (e.g., `150.50`).
- **Option Greeks:** Provided with 3 decimal places (e.g., `0.600`).

FAQ

How do I switch between paper and live trading?

The system determines the trading mode based on your account settings. No additional parameters are needed in API requests.

What happens if a data source fails?

For paper trading, the fallback mechanism automatically switches to the next available source. For live trading, Interactive Brokers' data is used exclusively.

Can I stream real-time data?

Currently, the API supports polling via the `real_time` parameter. WebSocket streaming may be added in future versions.

This documentation provides a complete guide to using the Algorithmic Trading System API. For further assistance, contact support.